
Aztec DeFi Bridge — All Options

Aztec Technical Challenge · Full Solution Document

Project	solidity/ — Aztec Connect DeFi Bridge
Option 1	Enhanced UniswapBridge (token↔token, slippage, pair-exists check)
Option 2	AaveLendingBridge (virtual-asset positions, async withdrawal)
Option 3	NettingBridge (ETH/TOKEN netting, async finalise)
Tests	✓ 7 / 7 passing (Option 1 fully tested on in-process Hardhat chain)

1. The Aztec Bridge Model

A DeFi bridge is a Layer-1 Solidity contract that exposes a DeFi protocol through a standard interface the Aztec rollup understands. Aztec batches many users' private transactions by bridge ID, then makes a single on-chain call that amortises gas over all participants.

1.1 Interface

```
interface IDefiBridge {
    // Synchronous or async DeFi interaction.
    function convert(
        Types.AztecAsset[4] calldata assets,
        uint64 auxData,           // 64-bit bridge-specific config
        uint256 interactionNonce,
        uint256 inputValue
    ) external payable
        returns (uint256 outputValueA, uint256 outputValueB, bool isAsync);

    // Returns true when async interaction is ready to finalise.
    function canFinalise(...) external view returns (bool);

    // Completes async interaction; transfers output assets to proxy.
    function finalise(...) external payable
        returns (uint256 outputValueA, uint256 outputValueB);
}
```

IDefiBridge.sol — every bridge must implement this interface

1.2 Asset Types

Type	Value	Meaning
ETH	1	Native Ether
ERC20	2	Any ERC-20 token; address in erc20Address field
VIRTUAL	3	Aztec position token; id = interactionNonce of the open call
NOT_USED	0	Slot unused for this interaction

1.3 Synchronous vs Asynchronous

- **Synchronous:** `convert()` returns output assets immediately (`isAsync = false`). The proxy verifies balances changed by exactly `outputValueA / outputValueB`.
- **Asynchronous:** `convert()` returns `isAsync = true` with both output values = 0. The rollup later polls `canFinalise()` and calls `finalise()` when ready.

1.4 Bridge ID Encoding

```
BridgeId (248 bits)
( auxData || bitConfig || outputAssetB || outputAssetA || inputAssetB || inputAssetA || bridgeAddressId
  64          32          30          30          30          30          32
```

```
BitConfig (32 bits)
( unused || firstAssetVirtual || secondAssetValid || secondAssetVirtual )
  29           1             1             1
```

The bridge ID encodes everything Aztec needs to route a transaction. All users with the same bridge ID are batched together into one L1 call, sharing gas costs proportionally.

2. Option 1 — Enhanced UniswapBridge

The original bridge only handled ETH↔ERC20 swaps with no slippage control and no pair-existence check. Three improvements were implemented.

2.1 Token-to-Token Swaps (ERC20 → ERC20)

Uniswap V2 has no direct ERC20↔ERC20 pairs for arbitrary tokens. The standard pattern routes through WETH as an intermediary: Token A → WETH → Token B.

```

    } else if (
        assets[0].assetType == Types.AztecAssetType.ERC20 &&
        assets[2].assetType == Types.AztecAssetType.ERC20
    ) {
        address[] memory path = new address[](3);
        path[0] = assets[0].erc20Address;    // Token A
        path[1] = weth;                      // intermediate WETH
        path[2] = assets[2].erc20Address;    // Token B

        _requirePairExists(path[0], path[1]);
        _requirePairExists(path[1], path[2]);

        IERC20(tokenIn).approve(address(router), inputValue);
        amounts = router.swapExactTokensForTokens(
            inputValue, amountOutMin, path, defiBridgeProxy, deadline
        );
        outputValueA = amounts[amounts.length - 1];
    }

```

UniswapBridge.sol — ERC20→ERC20 branch in convert()

2.2 Slippage Control via auxData

The unused auxData parameter now carries the maximum acceptable slippage in **basis points** (1 BPS = 0.01%). Setting auxData = 0 disables the check (amountOutMin = 0). Setting auxData = 500 means "reject if output < 95 % of expected".

```

function _calcAmountOutMin(
    uint256 inputValue,
    address[] memory path,
    uint64 auxData
) internal view returns (uint256) {
    if (auxData == 0) return 0;    // no protection

    uint256[] memory expected = router.getAmountsOut(inputValue, path);
    uint256 expectedOut = expected[expected.length - 1];

    // Require at least (1 - auxData/10000) of expected output.
    return expectedOut.mul(uint256(10000).sub(auxData)).div(10000);
}

```

UniswapBridge.sol — _calcAmountOutMin helper

2.3 Pair Existence Check

Rather than letting Uniswap silently fail or behave unexpectedly with zero-liquidity pairs, the bridge queries the factory directly:

```
function _requirePairExists(address tokenA, address tokenB) internal view {
    address pair = IUniswapV2Factory(factory).getPair(tokenA, tokenB);
    require(pair != address(0), 'UniswapBridge: PAIR_DOES_NOT_EXIST');
}
```

Reverts immediately if no liquidity pool exists for the given pair

2.4 Test Results (7 / 7 passing)

#	Test	auxData	Pass
1	ETH → ERC20 (original)	0	✓
2	ERC20 → ETH	0	✓
3	ERC20 → ERC20 (via WETH)	0	✓
4	ETH → ERC20 with 5 % slippage guard	500	✓
5	ERC20→ERC20 with 1 % slippage guard	100	✓
6	Revert — no Uniswap pair exists	0	✓
7	Revert — incompatible asset types	0	✓

2.5 Complete convert() Logic

```
function convert(assets, auxData, , inputValue) {
    require(msg.sender == defiBridgeProxy);
    isAsync = false;

    if (ETH → ERC20) {
        path = [weth, tokenOut]
        _requirePairExists(weth, tokenOut)
        amountOutMin = _calcAmountOutMin(inputValue, path, auxData)
        outputValueA = router.swapExactETHForTokens{value}(amountOutMin, ...)[last]

    } else if (ERC20 → ETH) {
        path = [tokenIn, weth]
        _requirePairExists(tokenIn, weth)
        approve(router, inputValue)
        amountOutMin = _calcAmountOutMin(inputValue, path, auxData)
        outputValueA = router.swapExactTokensForETH(inputValue, amountOutMin, ...)[last]

    } else if (ERC20 → ERC20) {
        path = [tokenIn, weth, tokenOut]
        _requirePairExists(tokenIn, weth)
        _requirePairExists(weth, tokenOut)
        approve(router, inputValue)
        amountOutMin = _calcAmountOutMin(inputValue, path, auxData)
        outputValueA = router.swapExactTokensForTokens(inputValue, amountOutMin, ...)[last]

    } else { revert(INCOMPATIBLE_ASSET_PAIR) }
```

```
}
```

3. Option 2 — AaveLendingBridge

A new bridge that deposits assets into **Aave V2** to earn yield, using Aztec's virtual asset mechanism to represent the position and async completion to allow interest to accrue before withdrawal.

3.1 Design Rationale

- **Virtual assets** eliminate the need to mint ERC-20 receipt tokens, saving gas and keeping position data private inside Aztec.
- **Async flow** allows users to lock funds for an arbitrary period. Withdrawals via `finalise()` automatically collect principal + all accrued interest in one step.
- **aToken accounting**: Aave aTokens rebase continuously. Holding them in the bridge contract means the user automatically earns interest with no additional calls.

3.2 Asset Flow Diagram

[illegible]

3.3 Key Contract Structures

```

struct Position {
    address underlying;           // deposited token (address(0) = ETH)
    address aToken;              // Aave aToken received on deposit
    uint256 depositedAmount;     // principal
    bool    readyToFinalise;     // set true by close convert()
}

mapping(uint256 => Position) public positions;           // openNonce → Position
mapping(uint256 => uint256) public closeToOpenNonce;    // closeNonce → openNonce

```

3.4 Interest Accrual

Aave aTokens are rebasing tokens: their balance increases continuously as interest accrues. Since the bridge holds aTokens between open and close, calling `lendingPool.withdraw(asset, type(uint256).max, ...)` withdraws the entire aToken balance, which automatically includes all accrued interest. The user receives **more** than they deposited without any extra steps.

3.5 Supported Interaction Patterns

Pattern	Input A	Output A	Async?	Use case
1→4 (open)	ERC20 / ETH	VIRTUAL	No	Deposit to Aave
2→1 (close)	VIRTUAL	ERC20 / ETH	Yes	Withdraw + interest

Without netting:

Leg A: swap 1 ETH → 1 992 TOKEN (full Uniswap trade)

Leg B: swap 1 800 TOKEN → 0.897 ETH (full Uniswap trade)

Savings: ~5 TOKEN for leg-A user, ~0.005 ETH for leg-B user

(fee saved = 0.3 % × 1 800 TOKEN worth of netting ≈ 5.4 TOKEN)

4.4 Async Flow

- `convert(ETH→TOKEN, nonce1)`: stores ETH amount, sets `hasPendingEthToToken = true`, returns `isAsync = true`.
- `convert(TOKEN→ETH, nonce2)`: triggers netting via `_netCaseA` / `_netCaseB`, marks both positions ready, returns `isAsync = true`.
- `canFinalise(nonce1)` and `canFinalise(nonce2)`: return true once both are ready.
- `finalise(nonce1)`: transfers TOKEN to DefiBridgeProxy.
- `finalise(nonce2)`: transfers ETH to DefiBridgeProxy.

4.5 Storage Layout

```
struct PendingInteraction {
    Direction direction; // ETH_TO_TOKEN or TOKEN_TO_ETH
    uint256    inputValue; // original input amount
    uint256    outputValue; // computed by _net(), transferred in finalise()
    bool       ready;      // true once _net() completes
}

mapping(uint256 => PendingInteraction) public pendingInteractions;

uint256 public pendingEthToTokenNonce; // nonce of the open ETH leg
bool    public hasPendingEthToToken;   // guard: prevents two ETH legs at once
```

5. Topics Covered

5.1 Solidity Patterns

- **Interface conformance:** all bridges implement `IDefiBridge`, enabling the proxy to call them generically.
- **Immutable state variables:** `defiBridgeProxy`, `weth`, `factory` are set once in the constructor and marked `immutable` for gas efficiency (~100 gas saved per read vs. storage slot).
- **SafeMath:** all arithmetic uses `SafeMath` to prevent overflow/underflow (required for Solidity < 0.8.0 which lacks built-in checks).
- **Storage cleanup:** `AaveLendingBridge` uses `delete` to zero out `Position` structs after finalisation, earning a gas refund and preventing stale-state bugs.
- **Stack-too-deep avoidance:** `NettingBridge._net` splits the netting into two internal functions (`_netCaseA`, `_netCaseB`) to stay within the EVM's 16-variable local stack limit.

5.2 DeFi Protocol Knowledge

- **Uniswap V2 AMM:** constant-product formula $x \cdot y = k$; all swaps touch WETH as the routing hub; `getAmountsOut/getAmountsIn` for price discovery.
- **Aave V2 lending:** rebasing `aTokens`; `deposit()/withdraw()` interface; `WETHGateway` for ETH positions; referral codes.
- **Slippage:** basis-point encoding, `amountOutMin` guard, price impact vs. fee cost trade-off.
- **Liquidity bootstrapping:** creating Uniswap pairs with `factory.createPair()` and low-level `pair.mint()` without the router (test setup).

5.3 Aztec Connect Concepts

- **Bridge IDs and batching:** same bridge ID → same rollup batch → amortised L1 gas across all participating users.
- **Virtual assets:** gas-efficient position tokens that live inside Aztec without an on-chain ERC-20 deployment; `id = interactionNonce`.
- **Asynchronous flow:** deferred settlement pattern for positions that need time to accrue (lending interest, option expiry, bridged L2 state).
- **Netting:** the optimal batching strategy for bilateral markets; reduces both Uniswap fees (0.3 % per swap) and price impact (especially relevant for large notionals).
- **auxData:** 64-bit opaque field allowing each bridge to encode its own configuration (slippage tolerance, token IDs, unlock timestamps, etc.).

5.4 Security Considerations

- **Caller validation:** every entry point checks `msg.sender == defiBridgeProxy` to prevent unauthorised calls that could drain the bridge.

- **Pair-existence guard:** `_requirePairExists` prevents sending ETH or tokens to zero-liquidity pools where recovery is uncertain.
- **Reentrancy:** the bridge sends ETH to the proxy via `.transfer()` (2 300 gas stipend) which prevents reentrancy into complex logic. Full reentrancy guards would be added in production.
- **Approval hygiene:** `approve(router, inputValue)` grants exactly the needed amount, never an unlimited allowance.

6. Summary

Option	Contract	Key features	Status
1	UniswapBridge.sol	ETH↔ERC20, ERC20↔ERC20 (via WETH), slippage guard (BPS), fair-exists ch	✓
2	AaveLendingBridge.sol	Aave v2 <deposit/withdraw, virtual asset position tokens, async interest accrual	✓
3	NettingBridge.sol	ETH↔TOKEN netting, Case A/B algorithm, async finalise, gas savings	✓

✓ All three options implemented | ✓ 7 / 7 Option-1 tests passing | ✓ All 14 contracts compile cleanly with Solidity 0.7.3
